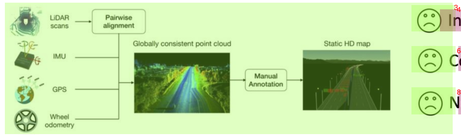
VectorMapNet: End-to-end Vectorized HD Map Learning

Yicheng Liu, Tianyuan Yuan, Yue Wang, Yilun Wang, Hang Zhao

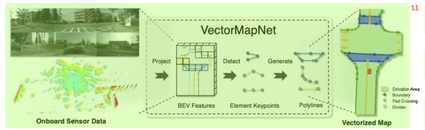
Motivations

Autonomous driving systems require an understanding of map elements on the road, including lanes, pedestrian crossing, and traffic signs, to navigate around the world.



- Involves a complex pipeline
- Costly and labor intensive
- Needs timely update

In contrast, we focus on using onboard sensors, including LiDARs and cameras, to estimate map elements on-the-fly.



- Simple and end-to-end pipeline
- Cheap and automatic
- Perceives dynamic environment

Contributions

- VectorMapNet is an end-to-end mapping approach.
- We utilize **polyline** to accommodate the heterogeneous nature of map elements.
- Our method achieves promising performance in online semantic HD map learning tasks.

Problem Formulation

Similar to HDMapNet, our task is to generate vectorized map elements using data from onboard sensors of autonomous vehicle, such as RGB cameras and/or LiDARs. These map elements include:

- *Lane dividers:* boundaries dividing lanes on the road, usually straight lines.
- *Road boundaries:* boundaries of roads separating roads and sidewalks, typically irregularly-shaped curves of arbitrary lengths.
- *Pedestrian crossings:* regions with white markings indicating legal pedestrian crossing points, typically represented as polygons.
- *Centerlines:* line segments that commonly used for driving direction, vehicle positioning, and navigation.

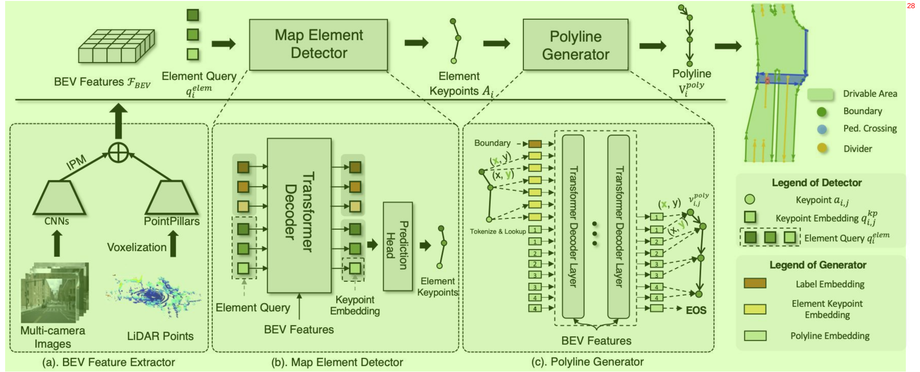
Why use polyline to represent maps?

Using polylines to represent map elements has three main advantages:

- Polylines are **a flexible primitive** that can represent complex geometric elements in HD maps.
- The order of polyline vertices is a natural way to encode **the direction of map elements**.
- The polyline representation has been **widely used by downstream** tasks, such as motion forecasting.

VectorMapNet

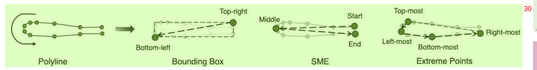
VectorMapNet, an end-to-end model, is designed to represent a *map* with a sparse set of *polylines* thus formulating the task as a sparse set detection problem. In our approach, we divide the task into three distinct components



(1) A **BEV Feature Extractor (A variant of IPM)** that lifts various sensor modality inputs into a canonical feature space (BEV space).

(2) A **map element detector (DETR-like Transformer decoder)** that locates and classifies all map elements by predicting element keypoints A_i and their class labels l_i.

(3) A **Polyline Generator (Vanila Transformer decoder)** that produces a sequence of ordered polyline vertices which describes the local geometry of each detected map element A_i, l_i.



Three different keypoint representations are proposed here.

VectorMapNet for Motion Forecasting

To evaluate the capacity of our method and to investigate its usefulness in downstream tasks, we put our predicted HD map to the test within a motion forecasting task.

Motion Forecasting heavily relies on precise map information for accurate prediction of future motion.

Model Inputs	minADE ↓	minFDE↓	MR@2m↓
Traj.	0.909	1.577	19.6
Traj. + G.T. Map	0.779	1.390	18.0
Traj. + Pred. Map	0.826	1.477	18.2

Quantitative Results

We report the average precision that uses Chamfer distance and Fréchet Distance as the threshold.

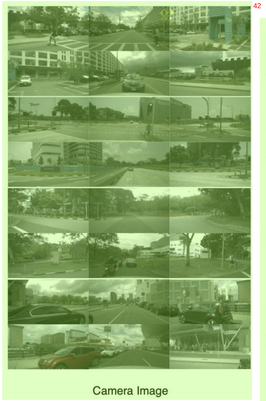
❑ Results on nuScenes

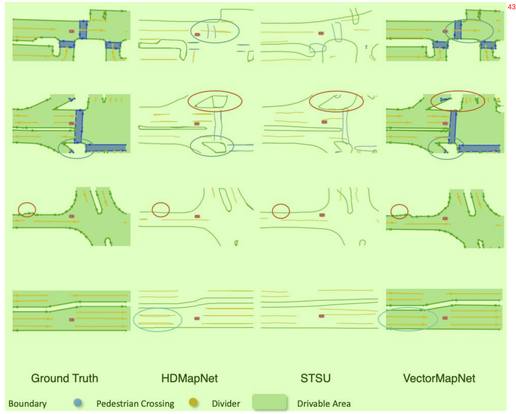
Methods	AP_{ped}	$AP_{divider}$	$AP_{boundary}$	mAP
STSU (Can et al., 2021)	7.0	11.6	16.5	11.7
HDMapNet (Camera) (Li et al., 2021)	14.4	21.7	33.0	23.0
HDMapNet (LiDAR) (Li et al., 2021)	10.4	24.1	37.9	24.1
HDMapNet (Fusion) (Li et al., 2021)	16.3	29.6	46.7	31.0
VectorMapNet (Camera)	36.1	47.3	39.3	40.9
VectorMapNet (Camera) + fine-tune	42.5	51.4	44.1	46.0
VectorMapNet (LiDAR)	25.7	37.6	38.6	34.0
VectorMapNet (Fusion)	37.6	50.5	47.5	45.2
VectorMapNet (Fusion) + fine-tune	**48.2**	**60.1**	**53.0**	**53.7**

❑ Results on Argoverse2

		Fréchet Distance				Chamfer Distance			
Keypoint Representaion	$\#dim$	AP_{ped}	$AP_{divider}$	$AP_{boundary}$	mAP	AP_{ped}	$AP_{divider}$	$AP_{boundary}$	mAP
HDMapNet (Camera) (Li et al., 2021)	2	-	-	-	-	13.1	5.7	37.6	18.8
VectorMapNet (Camera)	2	43.2	45.5	52.0	46.9	38.3	36.1	39.2	37.9
VectorMapNet (Camera)	3	41.7	42.3	49.9	44.6	36.5	35.0	36.2	35.8

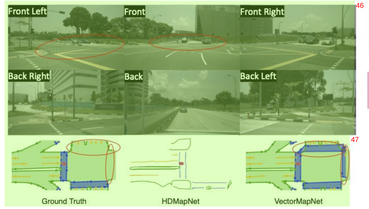
Qualitative Results





Using polylines as primitives has brought us two benefits compared with baselines:

- Polylines effectively encode the detailed map geometries, *e.g.,* the corners of boundaries (red ellipses).
- Polyline representations prevent Model from generating ambiguous results, as it consistently encodes direction information. In contrast, Rasterized methods are prone to falsely generating loopy curves (blue ellipses).

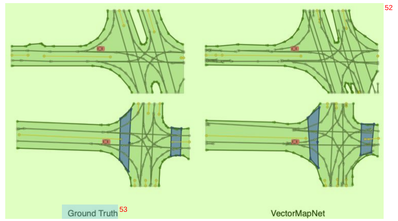


A benefit of of posing map learning as a detection problem:

The model can identify pedestrian crossings at intersections that are missed in the annotations of the HD map provided by the dataset.

Centerline Generation

To demonstrate the flexibility of polyline, we expand VectorMapNet to predict centerlines.



Reference

HDMapNet: An Online HD Map Construction and Evaluation Framework, Li et al., ICRA2022
Structured bird's-eye-view traffic scene understanding from onboard images, Can et al., ICCV 2021